Ahmed Younis at ArGuard Shared Tasks: Calibration Is Most of What Is Left to Tune
Decode-Time Structure and Threshold Transport for Arabic Harmful-Content Detection

Ahmed Younis
Minerva University
bakr@uni.minerva.edu

Abstract

We describe our systems for all four ArGuard 2026 subtasks: binary and fine-grained Arabic hateful-meme detection (A1/A2) and binary and granular harmful-prompt detection (B1/B2). Our systems are deliberately conventional (a LoRA-tuned Qwen3-VL-8B for memes, a nine-encoder ensemble for prompts) because our central finding is that, once a competent model was in place, most of our further gains came from decode-time calibration, much of it enabled by published priors and scoring feedback: per-class thresholds refit under structural constraints, rank-based decoding against known or published class priors, and threshold transport under distribution shift. On subtask A2, our submissions rose from 0.248 (dev-test) to 0.419 (blind test) macro-F1: switching to the VLM added 0.096, decode-time changes 0.068 (0.043 of it from re-decoding against class support revealed by test-phase scoring feedback) and averaging more training runs 0.025 (partly confounded with a decode-multiplier change), while moving to the blind test cost 0.017; a fixed-model ablation on a char-ngram model attributes +0.070 to the decode stack alone. On B1, recalibrating a single threshold recovered +0.16 macro-F1 after we detected that the blind test was ≈49% unsafe against an 80% unsafe training prior. Our final systems ranked **1st on subtask A2** (macro-F1 0.4194; 0.3739 before test-phase per-class scoring feedback), 3rd on A1 (0.7889), 3rd on B2 (0.7695), and 7th on B1 (0.7552). We additionally report a benchmark audit conducted during participation: a sequential-id label leak in the Task B development release (macro-F1 1.000 from ids alone), identified before the test release, left unexploited, and absent from the re-indexed test files; latent taxonomy structure that contradicts the task description; and a quantified gap between development and test distributions.

1 Introduction

The ArGuard shared task at ArabicNLP 2026 (Alam et al., 2026) targets two Arabic safety problems: multimodal hateful-meme detection over the AHA-Memes corpus (Kmainasi et al., 2026) (subtasks A1 binary, A2 fine-grained multi-label) and harmful-prompt detection for LLMs (B1 binary safe/unsafe, B2 seven-way harm domain). All subtasks rank by macro-F1.[1]

Shared-task papers usually foreground architecture. Our experience ran the other way: with competent but unremarkable models, most of what we could still tune lay in *decisions made after the forward pass*: which thresholds, fitted on which scores, against which prior. We therefore organize this paper around three calibration mechanisms and one benchmark audit:

- **Structure-aware threshold refitting** (A2): the binary label constrains the fine-grained label space (hateful and non-hateful memes draw on disjoint label sets apart from the shared Other); exploiting this helps *only* if thresholds are refit on the constrained scores (§4.1).
- **Rank decoding against known priors** (A1/A2): the dataset card publishes the blind-test class balance; per-class scoring feedback reveals support. Decoding by rank against these counts replaces noisy threshold fits (§4.2).
- **Threshold transport under shift** (B1): the blind test is ≈49% unsafe against 80% in train; re-pinning one threshold to the new prior recovers +0.16 macro-F1, while adding balanced-retrained models did not help; the

[1]This paper reports as team *Ahmed Younis*; on the Codabench leaderboards the team name and username are `ahmedbakr`. Official final runs: A1 submission 881206, A2 881205, B1 881203, B2 881204. Code: `github.com/itsbakr/arguard-2026`.

remaining 0.20 gap to dev is not a threshold effect (§4.3).

- **A benchmark audit** (§6): a complete label leak in the Task B development ids, identified pre-test-release, left unexploited, and absent from the re-indexed test files; latent taxonomy structure (§2); and near-duplicate pairs across train and dev that, contrary to the usual alarm, shift dev scores by at most 0.3 points under grouped cross-validation.

2 Data and Preprocessing

Task A (memes). We used only the official AHA-Memes release (Kmainasi et al., 2026): 3,500 train / 500 dev / 500 dev-test / 500 blind test memes with OCR text. No external data: we measured the permitted alternatives and rejected them. Prop2Hate-Meme (Alam et al., 2024) is single-label LLM-silver with $\kappa \approx 0.4$ against humans; it shares no filenames with ArGuard (0/3,053), although its parent corpus ArMeme shares at least 10 memes with ArGuard train, dev and dev-test (1 filename, 9 identical OCR texts). Images were capped at 1024^2 pixels for both training and inference; the OCR `text` field, visibly noisy, was appended to the prompt as an auxiliary channel.

Task B (prompts). The official CSVs: 16,835 train + 2,405 dev rows (prompt, dialect, binary and granular labels). Two rows carry empty labels and were dropped. The dialect column, available at test time, was prepended as a literal bracketed token holding its original Arabic value (e.g. the Arabic word for 'Egyptian'; `[UNK]` when the field is blank). We evaluated AraSafe (Mubarak et al., 2025) as permitted external data and measured zero prompt overlap with ArGuard but an inverted class balance in its human-written split (89.6% safe, against 79.8% unsafe in ArGuard train; its synthetic split is entirely unsafe) and an incompatible taxonomy, so we did not use it for label supervision.

Three structural facts drove our design. (i) The A2 label space is constrained by the A1 label: hateful memes carry only {Mocking, Incitement, Slurs, Dehumanization, Contempt, Inferiority, Exclusion, Other}; non-hateful only {Sarcasm, Humor, Other}. (ii) The B2 granular label populates *safe* rows as well (safe rows take only 4 of 7 categories, and `Bully`, `FraudandDeception` and `hate_speech` are 100% unsafe), so B2 is a seven-way harm-domain task over all rows, although the task description says "given an unsafe prompt". We verified against the platform: our all-rows offline scorer reproduced official scores to four decimal places. (iii) The dataset card publishes the blind-test binary balance for Task A (148 Hateful / 352 Not Hateful; 29.6% vs. 37.8% in every public split).

3 System Architecture and Training

Task A: one vision-language model (VLM), two heads. We LoRA-tune (Hu et al., 2022) `QCRI/MemeLens-VLM` (Shahroor et al., 2026),[2] a Qwen3-VL-8B-Instruct (Qwen Team, 2025) checkpoint released by the organizers' group and already adapted to Arabic meme understanding, with r=32, α=64, dropout 0.05 on all LLM attention and MLP projections, vision tower frozen. MemeLens's own training mix includes Prop2Hate-Meme and ArMeme but not AHA-Memes, whose release postdates the checkpoint; exposure of the backbone to the small number of ArGuard memes that also appear in ArMeme, of which Prop2Hate-Meme is a subset (§2), under a different taxonomy, therefore cannot be excluded. Rather than generative readout, a linear head over the last non-padding hidden state emits 1 binary + 10 sigmoid logits, trained jointly with class-weighted binary cross-entropy (weight $\sqrt{\text{neg}/\text{pos}}$ clipped to $[1, 8]$ for the ten A2 logits; neg/pos for the binary logit). lr 1e-4 cosine, effective batch 16, 3 epochs, bf16; ∼50 min per run on one H100. Final submissions average probabilities over 8 runs, all trained on train only: seeds 42–46, plus a second run of seeds 42–44. The three repeat runs were meant to add the dev split but did not (the setting never reached the remote training job, and each logged 657 optimizer steps, the train-only count, against 750 for train+dev), so they differ from the first runs only through run-to-run nondeterminism.

Task B: encoder ensemble. Nine encoders (MARBERTv2 (Abdul-Mageed et al., 2021), AraBERTv02-twitter (Antoun et al., 2020), CAMeLBERT-mix (Inoue et al., 2021) × 3 seeds), each with a shared backbone and two heads (binary + seven-way) trained multi-task (lr 2e-5, 3 epochs, max_len 96); probabilities averaged. For B2 we blend at equal weight with a character 3–5-gram TF-IDF logistic model, which alone reaches dev macro-F1 0.86; the blend lifts dev B2 macro-

[2] `huggingface.co/QCRI/MemeLens-VLM`

Decode rule (model fixed)	macro-F1
global 0.5 threshold	0.2559
global 0.5 + A1-gating	0.1972
per-class thresholds	0.3061
per-class, fit ungated, applied gated	0.2881
per-class, refit on gated scores	0.3123
+ rare-class floors (1.5× prior)	**0.3256**

Table 1: A2 decode ablation on held-out dev (char-ngram scores). A structural constraint imposed without refitting *reduces* macro-F1.

F1 from 0.8997 (encoders alone) to 0.9064. B2 is decoded by argmax over the blended seven-way probabilities. Because the metric is computed over all rows rather than only unsafe ones (§2), we train on all rows, which changes both the training population and the class prior; we did not measure an unsafe-only alternative. A blind-test probe that divided the blended probabilities by the training class prior before the argmax (with the char-ngram member refit on train+dev) scored 0.7621, below the submitted argmax's 0.7695.

4 Decode-Time Calibration

4.1 Structure-aware refitting on A2

A2 macro-F1 averages ten *independent* binary columns (`zero_division=0`): a false positive on a rare class costs nothing elsewhere, each class is worth 0.1 of the metric, and an abstained class scores 0. A global 0.5 threshold therefore silently abandons the tail (Exclusion: 10 train positives). Table 1 shows our held-out ablation with the model fixed. Two results matter. First, per-class F1-optimal thresholds are worth +0.05. Second (the subtlety), gating scores by the A1 constraint *alone hurts* (0.256→0.197): the constraint shrinks the gated classes' scores (labels predicted at 0.5 fall from 448 to 202) and invalidates thresholds fitted on unconstrained scores (per-class thresholds fit ungated, applied gated: 0.3061→0.2881). Refitting thresholds *on the gated scores* makes the same constraint profitable, and per-class floors at 1.5× the train prior recover rare-class recall. This decode stack took the identical model from 0.2559 to 0.3256 on held-out dev (+0.070). We frame the A1→A2 constraint as hierarchical constrained decoding, following the label-hierarchy treatment of Dimitrov et al. (2024).

4.2 Rank decoding against known counts

Where a class's true count is known or estimable, we replace thresholds with top-k rank decoding. For A1 the dataset card's published test balance (148/352) sets k: our submission at top-148 achieved hateful precision = recall = 0.703 exactly, confirming the published prior. For A2, per-class scoring feedback during the test phase reveals gold support; re-decoding with k set to 1.1× the revealed support (at least 3) raised test macro-F1 from 0.3739 to 0.4167 in a single resubmission (Contempt, whose test support of 36 was 2.6× its single-annotator dev-test support of 14, consistent with the dataset card's union-of-matching-annotators test protocol, gained +0.247 F1). We discuss the propriety of this channel in §6.

4.3 Threshold transport under shift on B1

Our first B1 test submission applied a dev-tuned threshold (0.35, tuned on the char-ngram member) to the encoder average and scored 0.5959, far below the ensemble's dev macro-F1 of 0.954. Backing the implied balance out of the platform's accuracy reading (0.6417 accuracy at 84.4% predicted unsafe) indicated the blind test is ≈49% unsafe versus 80% in train, assuming near-total unsafe recall at that operating point (which held on dev): a large prior shift. Re-pinning the *same ensemble's* threshold so that 49% of test prompts are labeled unsafe (its 51st-percentile score) scored 0.7552 (+0.16), with accuracy, precision, recall and macro-F1 all equal to three decimals, the signature of a matched operating point. Retraining the ensemble with balanced oversampling barely changed its ranking (Spearman 0.946 against the original), and a rank-averaged 18-model variant (the original nine plus nine balanced) scored *lower* (0.7380), so the prior mismatch itself is a threshold problem: here, that one dev-tuned constant cost our system 15.9 macro-F1 points (0.5959 vs. 0.7552). The threshold is not the whole gap, though: at the matched operating point the ensemble still trails its dev score by 0.20 (0.7552 vs. 0.954).

5 Experimental Results

Table 2 summarizes official results. The A2 trajectory separates mostly into model and decode steps (the 5-seed step changed both). On dev-test: 0.2482 (char-ngram; per-class thresholds fit on ungated scores, applied to gated scores) → 0.2690

Subtask	Ours	Rank	Top score
A2 fine-grained meme	0.4194	**1st**	0.4194
A1 binary meme	0.7889	3rd	0.823
B2 harm domain	0.7695	3rd	0.7903
B1 safe/unsafe	0.7552	7th	0.9840

Table 2: Official blind-test macro-F1 and final ranks (leaderboards of 19 Aug 2026). Field sizes: 16 (A1), 11 (A2), 22 (B1), 17 (B2) systems. Official baselines: 0.678 (A1), 0.295 (A2), 0.592 (B1), 0.687 (B2).

(refit on gated scores + floors) $\rightarrow$ 0.3650 (VLM, one seed, same decode rule) $\rightarrow$ 0.3694 (rank decode, 1.25$\times$ known support) $\rightarrow$ 0.3913 (5-seed average, 1.15$\times$). On the blind test: 0.3739 (same 5 seeds, estimated support) $\rightarrow$ 0.4167 (1.1$\times$ revealed support) $\rightarrow$ 0.4194 (8 runs). Switching to the VLM with the decode rule fixed added +0.096; decode changes added +0.068 (+0.021 on the char-ngram model, +0.004 and +0.043 on the VLM, the last using test-phase feedback); averaging more runs added +0.025 (the 5-seed step also tightened the rank-decode multiplier from 1.25$\times$ to 1.15$\times$); moving from dev-test to the blind test cost $-$0.017. After the switch to the VLM, decoding outweighed further model-side work by about 1.9 to 1 (0.047 vs. 0.025), mostly through the feedback-based support correction. Both teams that outrank us on A1 (0.823 and 0.790 vs. our 0.789) trail our final A2 (0.390 and 0.360 vs. 0.419), although our pre-feedback 0.374 would sit below the first of them.

On B1, the two systems above 0.98 sit six points clear of the next system (0.924). Engineering notes for reproducibility: a torch 2.9.x regression, consistent with the upstream Conv3d report (pytorch#174051), makes Qwen3-VL training steps with images $\sim$60–76$\times$ slower (10.69–13.76s vs. 0.18s per two-image forward+backward step after pinning torch 2.10); PEFT with gradient checkpointing can leave LoRA adapters with no gradient (None) unless input gradients are enabled, and torch warns only once, so we guard it with a hard assertion at step 0. Total compute: about 9 H100-hours (eight $\sim$50-minute LoRA runs plus debugging and inference); Task B encoders ran on A10 GPUs.

6 Benchmark Audit

Sequential-id leakage (Task B, dev release). Sorting the 19,240 released rows by integer id yields seven perfectly contiguous granular-label runs (e.g. adult_content occupies ids 0–6560) and seven binary runs; the two malformed empty-label rows sit exactly on run boundaries, fingerprinting per-domain concatenation before splitting. A nearest-known-id lookup fitted on train only, reading zero characters of text, scores macro-F1 **1.0000** on dev for both B1 and B2; shuffling ids collapses it to 0.51/0.15. Of the 4,813 ids in $[0, 24050]$ without a usable label (the 4,811 absent from the released files plus the two empty-label rows; the presumed blind-test pool at release time), 99.7% resolve unambiguously. We identified this on 29 July, before the test release, and did not act on it: we submitted no id-derived predictions. The released test files are re-indexed from 1 and independent of the development ids, so the artifact does not affect evaluation-phase scores. We report it here, with a reproduction script in our code release, so that future releases can block-randomize or hash ids before splitting.

Development sets overstate. The dev-to-test drops (B1 0.954$\rightarrow$0.755, B2 0.906$\rightarrow$0.770) are not explained by near-duplicate construction: 169 near-duplicate prompt pairs (Jaccard $\geq$0.8, 100% label agreement) span train and dev, yet grouped cross-validation shifts measured dev scores by $\leq$0.3 points (the standard alarm overstates); the test distribution shift does not appear in dev at all.

Feedback as a calibration channel. Per-class scoring output reveals gold support (§4.2). No task rule addresses submission-time use of scoring feedback; we used it openly within the posted submission limits, disclose it here, and report the A2 score obtained before any test-phase feedback (0.3739, whose support estimates drew on dev-phase feedback) alongside the final 0.4194 so systems remain comparable. We recommend that future editions aggregate per-class diagnostics or withhold them until the phase closes.

7 Conclusion

Conventional models with careful decoding placed 1st of 11 systems on fine-grained meme classification (A2), 3rd of 16 on A1, 3rd of 17 on B2, and 7th of 22 on B1. The general lesson for long-tail safety classification is that, once a competent model is in place, much of macro-F1 is a property of the *decision layer*: thresholds must be fitted on the scores actually used at inference, transported to the target prior, and floored on rare classes; and benchmark artifacts (id leakage, latent taxonomy

structure, distribution shift) can dominate modeling effort in both directions. Our concurrent ImageEval 2026 systems reach the same conclusion for judge-weight calibration in multimodal evaluation (Younis, 2026). Future work: agreement-aware training against adjudicated test protocols, and organizer-side randomization checks as release practice.

Limitations

Our A1/A2 decode analysis is measured on 500-row dev, dev-test and test sets, where 0.02 macro-F1 is roughly 7–10 A1 predictions but, on A2, anywhere from a single Exclusion label to about 40 Sarcasm labels; the B1 threshold-transport result rests on a single pair of test submissions. We did not measure seed-to-seed variance. Gaps of 0.001–0.003 to adjacent systems (A1 against both neighbors, B2 against fourth place) are as small as the 0.003 that going from 5 to 8 runs moved our own A2 score, so those ranks should not be read as real differences. The B1 balance estimate (49%) is inferred from scoring feedback, not released statistics. Our decode-time calibration is language-agnostic, but our backbones are not (Arabic-pretrained encoders for Task B; for Task A, a multilingual meme VLM whose training includes Arabic memes); we did not evaluate cross-lingual transfer. The support-corrected A2 decode depends on per-class feedback that other participants may not have used, which complicates strict system-to-system comparison.

Ethical Considerations

Both tasks involve intentionally offensive content; we handled it for research evaluation only, did not redistribute images or prompts, and used the Task A data under its CC BY-NC 4.0 license and the Task B data under the shared task's research-use terms. Better harmful-content classifiers can be repurposed for censorship; per-dialect behavior differences (train safe-rates range from 9.2% for Egyptian to 30.1% for Syrian Arabic) risk unequal moderation quality across dialect communities and should be audited before deployment. We did not exploit the identified label leak, and we document it here with a reproduction so that future releases can protect the integrity of all participants' results.

Acknowledgments

We thank the ArGuard organizers for running the task and the reviewers for their careful verification of our numbers.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*.

Firoj Alam, Md. Rafiul Biswas, Mohamed Bayan Kmainasi, Ali Ezzat Shahroor, Hamdy Mubarak, Georgios Mikros, Abul Hasnat, and Wajdi Zaghouani. 2026. ArGuard Shared Task: Harmful content detection in Arabic Memes and LLM Prompts. In *Proceedings of the Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Firoj Alam, Md. Rafiul Biswas, Uzair Shah, Wajdi Zaghouani, and Georgios Mikros. 2024. Propaganda to hate: A multimodal analysis of Arabic memes with multi-agent LLMs. In *Web Information Systems Engineering (WISE 2024)*.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools*.

Dimitar Dimitrov, Firoj Alam, Maram Hasanain, Abul Hasnat, Fabrizio Silvestri, Preslav Nakov, and Giovanni Da San Martino. 2024. SemEval-2024 task 4: Multilingual detection of persuasion techniques in memes. In *Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024)*.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*.

Mohamed Bayan Kmainasi, Ali Ezzat Shahroor, Abul Hasnat, Md. Rafiul Biswas, Wajdi Zaghouani, and Firoj Alam. 2026. AHA-Memes: A fine-grained multimodal benchmark for understanding hate in Arabic memes. *arXiv preprint arXiv:2607.27393*.

Hamdy Mubarak, Abubakr Mohamed, and Majd Hawasly. 2025. AraSafe: Benchmarking safety in Arabic LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 9976–9992.

Qwen Team. 2025. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*.

Ali Ezzat Shahroor, Mohamed Bayan Kmainasi, Abul Hasnat, Dimitar Dimitrov, Giovanni Da San Martino, Preslav Nakov, and Firoj Alam. 2026. MemeLens: Multilingual multitask VLMs for memes. *arXiv preprint arXiv:2601.12539*.

Ahmed Younis. 2026. Ahmed Younis at ImageEval 2026 shared tasks: The judge needs a prior and the task needs a constraint. In *Proceedings of the Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.